

Gender dynamics experiments

Eduardo García-Portugués, Carlos Vaquero

2026-08-03

Running experiment 1

Load libraries

```
# Load libraries
library(knitr)
library(dplyr)
library(recipes)
library(rsample)
library(yardstick)
library(glmnet)
library(ggplot2)
library(caret)
library(randomForest)
library(tibble)
sessionInfo()
```

R version 4.5.2 (2025-10-31)

Platform: aarch64-apple-darwin20

Running under: macOS Tahoe 26.6

Matrix products: default

BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/

LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK ve

locale:

[1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8

time zone: Europe/Madrid

tzcode source: internal

attached base packages:

[1] stats graphics grDevices utils datasets methods base

other attached packages:

```
[1] tibble_3.3.1      randomForest_4.7-1.2 caret_7.0-1
[4] lattice_0.22-9    ggplot2_4.0.3      glmnet_5.0
[7] Matrix_1.7-6      yardstick_1.4.0    rsample_1.3.2
[10] recipes_1.3.2     dplyr_1.2.1        knitr_1.51
```

loaded via a namespace (and not attached):

```
[1] gtable_0.3.6      shape_1.4.6.1      xfun_0.60
[4] vctrs_0.7.3       tools_4.5.2        generics_0.1.4
[7] stats4_4.5.2      parallel_4.5.2     ModelMetrics_1.2.2.2
[10] pkgconfig_2.0.3   data.table_1.18.4  RColorBrewer_1.1-3
[13] S7_0.2.2          lifecycle_1.0.5    stringr_1.6.0
[16] compiler_4.5.2    farver_2.1.2       codetools_0.2-20
[19] htmltools_0.5.9   class_7.3-23       yaml_2.3.12
[22] prodlim_2025.04.28 pillar_1.11.1      furrr_0.4.0
[25] tidyr_1.3.2       MASS_7.3-66        gower_1.0.2
[28] iterators_1.0.14  rpart_4.1.27       foreach_1.5.2
[31] nlme_3.1-170      parallelly_1.48.0  lava_1.8.2
[34] tidyselect_1.2.1  digest_0.6.39      stringi_1.8.7
[37] future_1.75.0     reshape2_1.4.5     purrr_1.2.2
[40] listenv_1.0.0     splines_4.5.2      fastmap_1.2.0
[43] grid_4.5.2        cli_3.6.6          magrittr_2.0.5
[46] survival_3.8-9    future.apply_1.20.2 withr_3.0.3
[49] scales_1.4.0      lubridate_1.9.5    timechange_0.4.0
[52] rmarkdown_2.31    globals_0.19.1     otel_0.2.0
[55] nnet_7.3-20       timeDate_4052.112  evaluate_1.0.5
[58] hardhat_1.4.3     rlang_1.3.0        Rcpp_1.1.2
[61] glue_1.8.1        pROC_1.19.0.1      ipred_0.9-15
[64] jsonlite_2.0.0    plyr_1.8.9         R6_2.6.1
```

Read dataset

```
# Load the single-source dataset, downloaded from Zenodo on first use. The file
# is not tracked by the repository; it is fetched once and reused thereafter.
# The literal NA values in Sex (arias with no singer record) become real NAs, and
# stringsAsFactors turns Gender/Sex and the four character-valued interval
# features into factors with alphabetical levels, so the second level is the
# masculine/male class in every experiment
data_file <- "sopranovoices.csv"
data_doi <- "10.5281/zenodo.21757127"
data_url <- paste0("https://zenodo.org/records/21757127/files/",
  "sopranovoices.csv?download=1")
data_md5 <- "2b495d57628d67f962e8fe44fdcfb02d"
if (!file.exists(data_file)) {
```

```

message("Downloading ", data_file, " from Zenodo (DOI ", data_doi, ")")
# Raise the 60 s default timeout for this transfer only, and write to a
# temporary file so an interrupted download never leaves a truncated
# sopranovoices.csv that later runs would silently reuse
old_opts <- options(timeout = max(600, getOption("timeout")))
tmp_file <- paste0(data_file, ".part")
download.file(url = data_url, destfile = tmp_file, mode = "wb")
options(old_opts)
if (!identical(unname(tools::md5sum(tmp_file)), data_md5)) {

  unlink(tmp_file)
  stop("Checksum mismatch for the file downloaded from ", data_url)

}
file.rename(tmp_file, data_file)
}

# Guarantee that every run uses the exact deposited file
stopifnot(identical(unname(tools::md5sum(data_file)), data_md5))
df_all <- read.csv(data_file, stringsAsFactors = TRUE)
stopifnot(nrow(df_all) == 1682, ncol(df_all) == 576)

# Metadata columns, never used as predictors
meta_cols <- c("AriaId", "ariaTitle", "AriaOpera", "Character", "Gender",
               "Composer", "Year", "Singer", "Sex")

# Experiment subsets and target y:
# 1 = all arias, y = character gender;
# 2 = arias whose character gender aligns with the singer's sex, y = gender;
# 3 = arias with a known singer, y = singer's sex
df <- switch(
  as.character(experiment),
  "1" = df_all |>
    mutate(y = Gender),
  "2" = df_all |>
    filter((Gender == "Feminine" & Sex == "Female") |
           (Gender == "Masculine" & Sex == "Male")) |>
    mutate(y = Gender),
  "3" = df_all |>
    filter(!is.na(Sex)) |>
    mutate(y = Sex),
  stop("Experiment ", experiment, " not supported (expected 1, 2 or 3)")
)

# Drop metadata and any factor levels emptied by the filters
df <- df |>
  select(-all_of(meta_cols)) |>

```

```

droplevels()

# Number of observations per group
df |>
  group_by(y) |>
  summarise(n = n(), .groups = "drop")

# A tibble: 2 x 2
   y         n
<fct>   <int>
1 Feminine  805
2 Masculine 877

# Split into train / test (70% / 30%), stratified by y so that both sets
# preserve the class balance of the full dataset.
set.seed(123)
split_obj <- initial_split(df, prop = 0.70, strata = y)
train <- training(split_obj)
test <- testing(split_obj)

```

Corpus vocal ranges

```

# Two-panel boxplots of the arias' vocal ranges: highest and lowest notes by
# premiering singer's sex (arias with a known singer) and by character gender
# (all arias). Independent of the current experiment, so it is rendered only
# in the experiment == 1 pass.
if (experiment == 1) {

  make_ranges <- function(data, group, panel) {
    bind_rows(
      tibble(group = group, note = "Highest notes",
              index = data$PartSop_HighestNoteIndex),
      tibble(group = group, note = "Lowest notes",
              index = data$PartSop_LowestNoteIndex)
    ) |>
    mutate(panel = panel)
  }
  singers <- df_all |> filter(!is.na(Sex))
  ranges_df <- bind_rows(
    make_ranges(singers, as.character(singers$Sex), "Singer"),
    make_ranges(df_all, as.character(df_all$Gender), "Character")
  ) |>
  mutate(panel = factor(panel, levels = c("Singer", "Character")))
}

```

```

# C-note axis: note index 60 = C4, so octave number = index / 12 - 1
c_breaks <- seq(48, 96, by = 12)
plot_ranges <- ggplot(ranges_df, aes(x = group, y = index, fill = note)) +
  geom_boxplot(outlier.shape = 1, outlier.size = 0.8) +
  facet_wrap(~ panel, scales = "free_x", strip.position = "bottom") +
  scale_fill_manual(values = c("Highest notes" = "grey35",
                              "Lowest notes" = "grey85")) +
  scale_y_continuous(breaks = c_breaks,
                    labels = paste0("C", c_breaks / 12 - 1),
                    limits = c(48, 96)) +
  labs(x = NULL, y = "Note", fill = NULL) +
  theme_bw() +
  theme(strip.placement = "outside",
        strip.background = element_blank(),
        axis.text.x = element_text(angle = 45, hjust = 1),
        legend.position = c(0.998, 0.995),
        legend.justification = c(1, 1),
        legend.key.size = grid::unit(0.85, "lines"),
        legend.text = element_text(size = 8),
        # Careful with randomForest::margin()
        legend.margin = ggplot2::margin(1, 3, 1, 1),
        legend.background = element_rect(fill = "white", colour = "grey60",
                                          linewidth = 0.3))

plot_ranges

# Save plot
dir.create("plots", showWarnings = FALSE, recursive = TRUE)
ggsave("plots/singers_vs_characters_ranges.png", plot = plot_ranges,
       width = 7.5, height = 2.9, dpi = 300, bg = "white")
}

```

Dataset summary tables

```

# Sources for the corpus-description tables of the paper: character gender vs.
# premiering singer's sex (Table 1) and the per-character breakdown (Table 2).
# Independent of the current experiment, so written only in the
# experiment == 1 pass.
if (experiment == 1) {

  dir.create("results/dataset_summary", showWarnings = FALSE, recursive = TRUE)

  # Table 1: distribution of arias by character gender and singer sex
  singer_col <- factor(
    ifelse(is.na(df_all$Sex), "Unknown singers",

```

```

    paste(df_all$Sex, "singers")),
  levels = c("Male singers", "Female singers", "Unknown singers")
)
table1 <- as.data.frame.matrix(table(df_all$Gender, singer_col)) |>
  rownames_to_column("Gender")
write.csv(table1, "results/dataset_summary/table1_distribution.csv",
  row.names = FALSE)
table1 |>
  kable()

# Table 2: per-character singer-sex breakdown
table2 <- df_all |>
  group_by(Character) |>
  summarise(
    Gender = first(Gender),
    female_ratio = ifelse(sum(!is.na(Sex)) > 0,
      sum(Sex == "Female", na.rm = TRUE) /
      sum(!is.na(Sex)), NA),
    male_ratio = ifelse(sum(!is.na(Sex)) > 0,
      sum(Sex == "Male", na.rm = TRUE) /
      sum(!is.na(Sex)), NA),
    known = sum(!is.na(Sex)),
    total = n(),
    .groups = "drop"
  ) |>
  arrange(desc(known), desc(total))
write.csv(table2, "results/dataset_summary/table2_characters.csv",
  row.names = FALSE)
table2 |>
  kable(digits = 2)
}

```

Character	Gender	female_ratio	male_ratio	known	total
Poro	Masculine	0.04	0.96	133	157
Didone	Feminine	0.82	0.18	132	152
Cleofide	Feminine	0.80	0.20	106	121
Enea	Masculine	0.11	0.89	103	116
Timante	Masculine	0.12	0.88	96	117
Dircea	Feminine	0.87	0.13	95	125
Arbace	Masculine	0.01	0.99	94	101
Mandane	Feminine	0.82	0.18	85	89
Erissena	Feminine	0.80	0.20	79	94
Selene	Feminine	0.82	0.18	76	89
Creusa	Feminine	0.86	0.14	59	73
Artaserse	Masculine	0.04	0.96	55	60
Gandarte	Masculine	0.06	0.94	53	63

Character	Gender	female_ratio	male_ratio	known	total
Semira	Feminine	0.75	0.25	52	53
Araspe	Masculine	0.15	0.85	47	52
Cherinto	Masculine	0.15	0.85	46	63
Megabise	Masculine	0.36	0.64	25	25
Alessandro	Masculine	0.00	1.00	23	28
Timagene	Masculine	0.21	0.79	19	23
Adrasto	Masculine	0.54	0.46	13	18
Iarba	Masculine	0.00	1.00	11	14
Osmida	Masculine	0.25	0.75	8	15
Artabano	Masculine	0.00	1.00	8	10
Demofoon	Masculine	0.50	0.50	6	10
Emirena	Feminine	1.00	0.00	5	5
Matusio	Masculine	0.25	0.75	4	5
Sabina	Feminine	1.00	0.00	3	3
Tamiri	Feminine	NA	NA	0	1

```
# Feature selection is name-based (deterministic given the column names)
selected_features <- select_sop_features(train)
stopifnot(length(selected_features) == 86)
train <- train |>
  select(all_of(selected_features), y)
test <- test |>
  select(all_of(selected_features), y)
dim(train)
```

```
[1] 1176 87
```

```
dim(test)
```

```
[1] 506 87
```

```
# Full dataset for the final-model recipe: all observations in their original
# order (train and test are a partition of df, but their concatenation would
# reorder the rows and hence change the CV folds of the final ridge fit)
combined <- df |>
  select(all_of(selected_features), y)
```

Preprocessing recipes

```
# Preprocessing recipe
preprocess_recipe <- function(data) {
  recipe(y ~ ., data = data) |>
```

```

# Filter and impute all predictors BEFORE dummifying.
step_zv(all_predictors()) |>
step_nzv(all_predictors()) |>
step_filter_missing(all_predictors(), threshold = 0.05) |>
step_impute_knn(all_predictors(), neighbors = 5) |>
# Collapse rare factor levels (BEFORE dummy creation)
step_other(all_nominal_predictors(), threshold = 0.10) |>
# Decorrelate numeric predictors only
step_corr(all_numeric_predictors(), threshold = 0.80) |>
step_lincomb(all_numeric_predictors(), max_steps = 20) |>
# Create dummy variables. With one_hot = TRUE every
# level surviving step_other() gets an indicator; dropping "_other" below
# then makes the pooled rare levels the reference
step_dummy(all_nominal_predictors(), one_hot = TRUE) |>
# Drop the dummies for the "other" level, making it the reference
step_rm(ends_with("_other")) |>
# Post-dummy cleanup
step_zv(all_predictors()) |>
step_nzv(all_predictors()) |>
step_lincomb(all_predictors(), max_steps = 20) |>
# Standardize LAST so the dummy indicators are standardized too and every
# column enters glmnet on unit-sd scale
step_normalize(all_numeric_predictors())
}

```

```

# Train recipe using train data
rec_train <- preprocess_recipe(data = train)
prep_rec_train <- prep(rec_train)

# Bake train data
train_bake <- bake(prep_rec_train, new_data = NULL)
test_train_bake <- bake(prep_rec_train, new_data = test)

# Train recipe using combined data (train + test)
rec_all <- preprocess_recipe(data = combined)
prep_all <- prep(rec_all)

# Fully baked dataset for the final model fit
all_bake <- bake(prep_all, new_data = NULL)

# Export datasets for future reference and use
dir.create("preprocessed_datasets", showWarnings = FALSE, recursive = TRUE)
write.csv(train_bake, paste0("preprocessed_datasets/train_bake_exp",
                             experiment, ".csv"),
          row.names = FALSE)
write.csv(test_train_bake, paste0("preprocessed_datasets/test_train_bake_exp",
                                  experiment, ".csv"),
          row.names = FALSE)

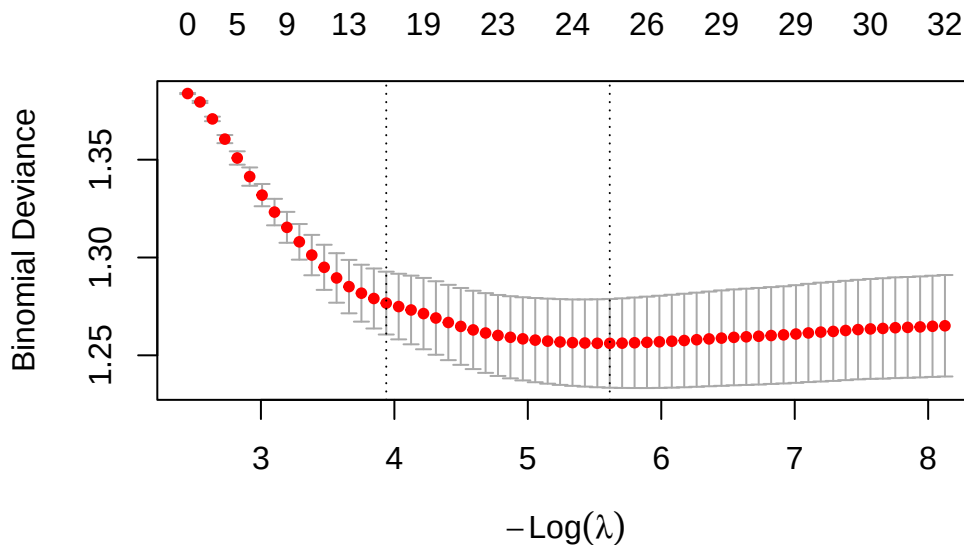
```

```
write.csv(all_bake, paste0("preprocessed_datasets/all_bake_exp", experiment,
                           ".csv"),
          row.names = FALSE)
```

Lasso logistic

Fitting

```
# Fit by 10-fold cross-validation, with folds stratified by y and shared
# between the lasso and ridge fits so that both models see the same partitions
set.seed(42)
train_bake_no_y <- train_bake |>
  select(-y) |>
  as.matrix()
foldid_train <- caret::createFolds(train_bake$y, k = n_folds, list = FALSE)
cv1_train <- cv.glmnet(x = train_bake_no_y, y = train_bake$y,
                      family = "binomial", alpha = 1, foldid = foldid_train)
plot(cv1_train)
```



Predictions

```
# Predictions
test_train_bake_no_y <- test_train_bake |>
  select(-y) |>
  as.matrix()
```

```

y_test_hat_LL <- predict(cv1_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "class")
p_test_hat_LL <- predict(cv1_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "response")

```

Metrics

```

# Metrics
acc_LL <- mean(test$y == y_test_hat_LL)
auc_LL <- auc_binary(test$y, p_test_hat_LL)
f1_LL <- f1(test$y, y_test_hat_LL)
data.frame("Accuracy" = acc_LL, "AUC" = auc_LL, "Macro_F1" = f1_LL) |>
  kable(digits = 4)

```

Accuracy	AUC	Macro_F1
0.6028	0.6692	0.6023

Relevant coefficients

```

# Extract nonzero coefficients for lambda.1se and order them by absolute value
coefs1_train <- predict(cv1_train, s = "lambda.1se", type = "coefficients") |>
  as.matrix() |>
  as.data.frame() |>
  rownames_to_column(var = "feature") |>
  rename("coefficient" = "lambda.1se")
coefs1_train |>
  filter(coefficient != 0) |>
  arrange(desc(abs(coefficient))) |>
  kable(digits = 4)

```

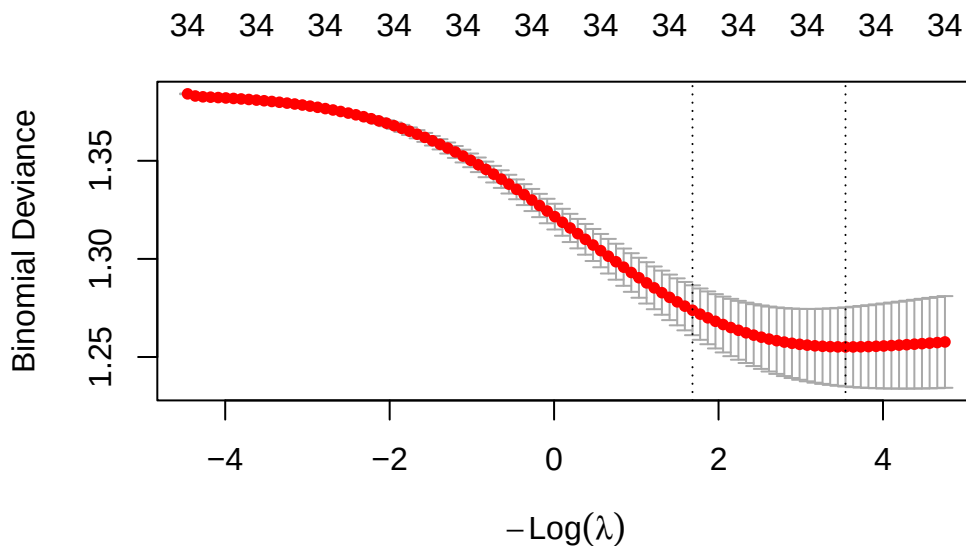
feature	coefficient
PartSop_HighestNoteIndex	-0.4326
PartSop_Ambitus	0.2897
PartSop_AverageDuration	0.1456
PartSop_IntervalsDiminishedDesc_Per	-0.1201
PartSop_VoicePresence	0.1197
(Intercept)	0.0977
PartSop_LargestSemitonesAsc	0.0900
PartSop_LargestSemitonesDesc	-0.0854
PartSop_IntervalsAugmentedDesc_Per	0.0750
PartSop_AscendingIntervallicMean	0.0616

feature	coefficient
PartSop_AbsoluteIntervallicTrimRatio	0.0566
PartSop_IntervalsPerfectDesc_Per	0.0484
PartSop_IntervalsMinorAll_Per	-0.0352
PartSop_IntervalsBeyondOctaveDesc_Per	0.0245
PartSop_IntervalsMajorAsc_Per	0.0174
PartSop_IntervalsDiminishedAsc_Per	0.0142

Ridge logistic model

Fitting

```
# Fit by 10-fold cross-validation, reusing the stratified folds of the lasso
cv0_train <- cv.glmnet(x = train_bake_no_y, y = train_bake$,
                      family = "binomial", alpha = 0, foldid = foldid_train)
plot(cv0_train)
```



Predictions

```
# Predictions
y_test_hat_RL <- predict(cv0_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "class")
p_test_hat_RL <- predict(cv0_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "response")
```

Metrics

```
# Metrics
acc_RL <- mean(test$y == y_test_hat_RL)
auc_RL <- auc_binary(test$y, p_test_hat_RL)
f1_RL <- f1(test$y, y_test_hat_RL)
data.frame("Accuracy" = acc_RL, "AUC" = auc_RL, "Macro_F1" = f1_RL) |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1
0.6087	0.6513	0.6084

Relevant coefficients

```
# Extract coefficients and sort them by absolute value
coefs0_train <- predict(cv0_train, s = "lambda.1se", type = "coefficients") |>
  as.matrix() |>
  as.data.frame() |>
  rownames_to_column(var = "feature") |>
  rename("coefficient" = "lambda.1se")
coefs0_train |>
  filter(coefficient != 0) |>
  arrange(desc(abs(coefficient))) |>
  head(30) |>
  kable(digits = 4)
```

feature	coefficient
PartSop_HighestNoteIndex	-0.2361
PartSop_IntervalsDiminishedDesc_Per	-0.1163
PartSop_AverageDuration	0.1129
PartSop_Ambitus	0.1116
PartSop_AscendingIntervallicMean	0.1061
PartSop_AbsoluteIntervallicTrimRatio	0.1018
PartSop_VoicePresence	0.0974
(Intercept)	0.0969
PartSop_LargestSemitonesAsc	0.0905
PartSop_IntervalsAugmentedDesc_Per	0.0880
PartSop_IntervalsPerfectDesc_Per	0.0799
PartSop_LargestSemitonesDesc	-0.0722
PartSop_IntervalsMinorAll_Per	-0.0721
PartSop_IntervalsDiminishedAsc_Per	0.0632
PartSop_IntervalsBeyondOctaveDesc_Per	0.0624

feature	coefficient
PartSop_IntervalsMajorAsc_Per	0.0613
PartSop_IntervalsPerfectAsc_Per	-0.0545
PartSop_SyllabicRatio	0.0489
PartSop_IntervallicTrimRatio	0.0412
PartSop_IntervallicTrimDiff	-0.0412
PartSop_LeapsDesc_Per	0.0408
PartSop_IntervallicMean	-0.0407
PartSop_IntervalsMajorAll_Per	0.0405
PartSop_LeapsAsc_Per	0.0405
PartSop_StepwiseMotionAll_Per	-0.0363
PartSop_IntervalsAugmentedAsc_Per	0.0346
PartSop_LargestIntervalAll_P8	-0.0241
PartSop_LargestIntervalAll_M9	0.0219
PartSop_IntervallicKurtosis	0.0202
PartSop_RhythmInt	-0.0155

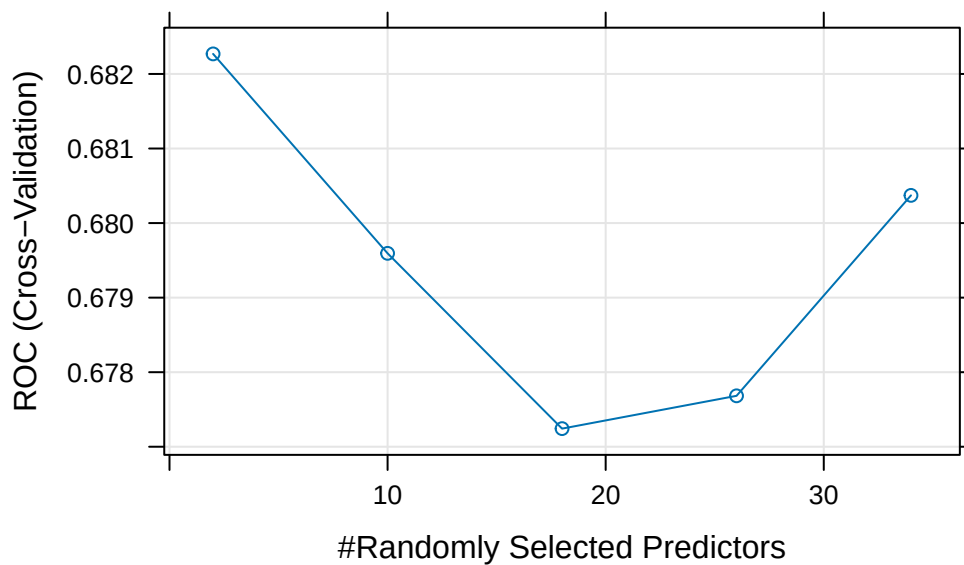
Random forests

Fitting

```
# Set seeds for reproducibility in caret with RF. caret expects a list of
# length n_folds + 1: one integer vector per fold (as long as the tuning grid)
# plus a final seed for the fit on the full training set.
set.seed(42)
rf_seeds <- c(
  lapply(seq_len(n_folds), function(i) rep(i, n_tune)),
  list(999L)
)

# Define the cross-validation scheme
cv_ctrl <- trainControl(method = "cv", number = n_folds, classProbs = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final", seeds = rf_seeds)

# Train Random Forest with tuning of mtry optimizing AUC
rf_train <- train(x = train_bake_no_y, y = train_bake$y, method = "rf",
  metric = "ROC", trControl = cv_ctrl, tuneLength = n_tune)
plot(rf_train)
```



Predictions

```
# Predictions
y_test_hat_RF <- predict(rf_train, newdata = test_train_bake_no_y)
positive_class <- levels(train_bake$y)[2] # "Class 1" is the second factor level
p_test_hat_RF <- predict(rf_train, newdata = test_train_bake_no_y,
                          type = "prob")[, positive_class]
```

Metrics

```
# Metrics
acc_RF <- mean(test$y == y_test_hat_RF)
auc_RF <- auc_binary(test$y, p_test_hat_RF)
f1_RF <- f1(test$y, y_test_hat_RF)
data.frame("Accuracy" = acc_RF, "AUC" = auc_RF, "Macro_F1" = f1_RF) |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1
0.6186	0.6676	0.618

Variables importance

```
# Variables importance
rf_importance <- varImp(rf_train)$importance
rf_importance |>
  rownames_to_column("feature") |>
  arrange(desc(Overall)) |>
  head(30) |>
  kable(digits = 4)
```

feature	Overall
PartSop_IntervalsPerfectDesc_Per	100.0000
PartSop_AscendingIntervallicMean	99.0607
PartSop_AbsoluteIntervallicTrimRatio	96.0270
PartSop_AverageDuration	94.1996
PartSop_VoicePresence	93.5079
PartSop_LeapsAsc_Per	87.5441
PartSop_IntervalsMinorAll_Per	87.0330
PartSop_SyllabicRatio	85.4795
PartSop_AscendingIntervals_Per	84.3657
PartSop_LeapsDesc_Per	83.3613
PartSop_IntervalsPerfectAsc_Per	83.1068
PartSop_IntervalsMajorAsc_Per	82.5382
PartSop_RhythmInt	82.3484
PartSop_IntervallicKurtosis	81.5901
PartSop_IntervalsPerfectAll_Per	81.3954
PartSop_IntervalsWithinOctaveDesc_Per	80.3199
PartSop_StepwiseMotionAll_Per	80.0017
PartSop_IntervallicMean	78.3152
PartSop_IntervallicTrimDiff	78.1632
PartSop_IntervalsMajorAll_Per	77.2295
PartSop_HighestNoteIndex	73.0579
PartSop_IntervallicTrimRatio	70.8549
PartSop_Ambitus	65.3795
PartSop_DottedRhythm	62.2788
PartSop_IntervalsDiminishedDesc_Per	60.9473
PartSop_IntervalsDiminishedAsc_Per	53.2375
PartSop_IntervalsBeyondOctaveAsc_Per	53.0902
PartSop_IntervalsAugmentedAsc_Per	52.7172
PartSop_LargestSemitonesAsc	52.6259
PartSop_LargestSemitonesDesc	46.0807

Metrics for all models on test data

```

# Save predictions for all models
dir.create("results/y_hats", showWarnings = FALSE, recursive = TRUE)
datasets_to_save <- list(
  RL = y_test_hat_RL,
  LL = y_test_hat_LL,
  RF = y_test_hat_RF,
  y_true = test$y
)
for (name in names(datasets_to_save)) {

  # Coerce to a one-column data frame with a stable column name. Note that
  # is.vector() is FALSE for both the glmnet class-prediction matrices and the
  # y_true factor, so the labels have to be extracted with as.vector().
  data_to_write <- data.frame(
    prediction = as.vector(datasets_to_save[[name]])
  )

  # Construct filename
  file_name <- paste0("results/y_hats/", name, "_", experiment, ".csv")

  # Write CSV
  write.csv(data_to_write, file_name, row.names = FALSE)

}

```

```

# Combine all metrics into a single table
metrics_table <- data.frame(
  Accuracy = c(acc_RL, acc_LL, acc_RF),
  AUC = c(auc_RL, auc_LL, auc_RF),
  Macro_F1 = c(f1_RL, f1_LL, f1_RF),
  Model = c("Ridge Logistic (RL)", "Lasso Logistic (LL)",
            "Random Forest (RF)")
)
metrics_table |>
  kable(digits = 4)

```

Accuracy	AUC	Macro_F1	Model
0.6087	0.6513	0.6084	Ridge Logistic (RL)
0.6028	0.6692	0.6023	Lasso Logistic (LL)
0.6186	0.6676	0.6180	Random Forest (RF)

Bootstrap tests for model comparison

```
# Run bootstrapping of accuracy with predictions
boot_RL <- boot_acc(test$y, y_test_hat_RL, n_boot = B, seed = 123)
boot_LL <- boot_acc(test$y, y_test_hat_LL, n_boot = B, seed = 123)
boot_RF <- boot_acc(test$y, y_test_hat_RF, n_boot = B, seed = 123)
boot <- list(RL = boot_RL, LL = boot_LL, RF = boot_RF)

# Bootstrap accuracy differences between RF and other models
diff_boot_RF_RL <- boot$RF - boot$RL
diff_boot_RF_LL <- boot$RF - boot$LL

# One-sided p-values (H1: RF better than other model)
p_RF_vs_RL <- (1 + sum(diff_boot_RF_RL <= 0)) / (B + 1)
p_RF_vs_LL <- (1 + sum(diff_boot_RF_LL <= 0)) / (B + 1)

# Two-sided confidence intervals for differences
ci_diff_RL <- quantile(diff_boot_RF_RL, probs = ci_probs)
ci_diff_LL <- quantile(diff_boot_RF_LL, probs = ci_probs)

# Add bootstrap tests to metrics table
metrics_table <- metrics_table |>
  mutate(
    ci_diff_lo = c(ci_diff_RL[1], ci_diff_LL[1], NA),
    ci_diff_up = c(ci_diff_RL[2], ci_diff_LL[2], NA),
    pval_better = c(p_RF_vs_RL, p_RF_vs_LL, NA)
  )
metrics_table |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1	Model	ci_diff_lo	ci_diff_up	pval_better
0.6087	0.6513	0.6084	Ridge Logistic (RL)	-0.0277	0.0474	0.3241
0.6028	0.6692	0.6023	Lasso Logistic (LL)	-0.0257	0.0573	0.2417
0.6186	0.6676	0.6180	Random Forest (RF)	NA	NA	NA

Comparison with majority class baseline

```
# Majority Class (MC) prediction baseline - always predict most common class
class_probs <- table(train$y) / length(train$y)
majority_class_train <- names(sort(class_probs, decreasing = TRUE))[1]
y_test_hat_MC <- factor(rep(majority_class_train, length(test$y)),
  levels = levels(test$y))
```

```

# Metrics for majority class baseline
acc_MC <- mean(test$y == y_test_hat_MC)
p_test_hat_MC <- rep(class_probs[majority_class_train], length(test$y))
auc_MC <- auc_binary(test$y, p_test_hat_MC)
f1_MC <- f1(test$y, y_test_hat_MC)

# Bootstrap accuracy for majority class baseline
boot_MC <- boot_acc(test$y, y_test_hat_MC, n_boot = B, seed = 123)

# Bootstrap accuracy differences between majority class baseline and models
diff_boot_RL_MC <- boot_RL - boot_MC
diff_boot_LL_MC <- boot_LL - boot_MC
diff_boot_RF_MC <- boot_RF - boot_MC

# One-sided p-values (H1: model better than majority class baseline)
p_RL_vs_MC <- (1 + sum(diff_boot_RL_MC <= 0)) / (B + 1)
p_LL_vs_MC <- (1 + sum(diff_boot_LL_MC <= 0)) / (B + 1)
p_RF_vs_MC <- (1 + sum(diff_boot_RF_MC <= 0)) / (B + 1)

# Two-sided confidence intervals for differences
ci_diff_RL_MC <- quantile(diff_boot_RL_MC, probs = ci_probs)
ci_diff_LL_MC <- quantile(diff_boot_LL_MC, probs = ci_probs)
ci_diff_RF_MC <- quantile(diff_boot_RF_MC, probs = ci_probs)

# Combine all metrics into a single table
metrics_table_MC <- data.frame(
  Accuracy = c(acc_RL, acc_LL, acc_RF, acc_MC),
  AUC = c(auc_RL, auc_LL, auc_RF, auc_MC),
  Macro_F1 = c(f1_RL, f1_LL, f1_RF, f1_MC),
  ci_diff_lo = c(ci_diff_RL_MC[1], ci_diff_LL_MC[1], ci_diff_RF_MC[1], NA),
  ci_diff_up = c(ci_diff_RL_MC[2], ci_diff_LL_MC[2], ci_diff_RF_MC[2], NA),
  pval_better = c(p_RL_vs_MC, p_LL_vs_MC, p_RF_vs_MC, NA),
  Model = c("Ridge Logistic (RL)", "Lasso Logistic (LL)",
            "Random Forest (RF)", "Majority Class (MC)")
)
metrics_table_MC |>
  kable(digits = 4)

```

Accuracy	AUC	Macro_F1	ci_diff_lo	ci_diff_up	pval_better	Model
0.6087	0.6513	0.6084	0.0257	0.1482	0.0027	Ridge Logistic (RL)
0.6028	0.6692	0.6023	0.0198	0.1403	0.0050	Lasso Logistic (LL)
0.6186	0.6676	0.6180	0.0375	0.1561	0.0010	Random Forest (RF)
0.5217	0.5000	0.3429	NA	NA	NA	Majority Class (MC)

```

# In a combined table
metrics_table_combined <- metrics_table_MC |>

```

```
mutate(ci_diff_lo_MC = ci_diff_lo, ci_diff_up_MC = ci_diff_up,
       pval_better_MC = pval_better) |>
mutate(ci_diff_lo_RF = c(metrics_table$ci_diff_lo, NA),
       ci_diff_up_RF = c(metrics_table$ci_diff_up, NA),
       pval_better_RF = c(metrics_table$pval_better, NA)) |>
select(Model, Accuracy, AUC, Macro_F1,
       ci_diff_lo_RF, ci_diff_up_RF, pval_better_RF,
       ci_diff_lo_MC, ci_diff_up_MC, pval_better_MC)
metrics_table_combined |>
  kable(digits = 4)
```

Model	Accu- racy	AUC	Macro_F1	ci_diff_lo_RF	ci_diff_up_RF	pval_bet- ter_RF	ci_diff_lo_MC	ci_diff_up_MC	pval_bet- ter_MC
Ridge	0.6087	0.6513	0.6084	-0.0277	0.0474	0.3241	0.0257	0.1482	0.0027
Logistic (RL)									
Lasso	0.6028	0.6692	0.6023	-0.0257	0.0573	0.2417	0.0198	0.1403	0.0050
Logistic (LL)									
Random	0.6186	0.6676	0.6180	NA	NA	NA	0.0375	0.1561	0.0010
Forest (RF)									
Majority Class (MC)	0.5217	0.5000	0.3429	NA	NA	NA	NA	NA	NA

```
# Save table of all metrics + bootstrap tests
dir.create("results/experiments", showWarnings = FALSE, recursive = TRUE)
write.csv(metrics_table_combined, paste0("results/experiments/all_results_",
                                         "Experiment_", experiment, ".csv"),
          row.names = FALSE)
```

```
# Print LaTeX table
latex_table <- metrics_table_combined |>
  mutate(ci_RF = paste0("$(", sprintf("%.4f", ci_diff_lo_RF), ", ",
                                   sprintf("%.4f", ci_diff_up_RF), ")$"),
         ci_MC = paste0("$(", sprintf("%.4f", ci_diff_lo_MC), ", ",
                                   sprintf("%.4f", ci_diff_up_MC), ")$"),
         p_RF = sprintf("%.4f", pval_better_RF),
         p_MC = sprintf("%.4f", pval_better_MC)) |>
  select(Model, Accuracy, AUC, Macro_F1, ci_RF, p_RF, ci_MC, p_MC)
latex_table |>
  kable(digits = 4)
```

Model	Accuracy	AUC	Macro_F1	ci_RF	p_RF	ci_MC	p_MC
Ridge Logistic (RL)	0.6087	0.6513	0.6084	(-0.0277, 0.0474)	0.3241	(0.0257, 0.1482)	0.0027
Lasso Logistic (LL)	0.6028	0.6692	0.6023	(-0.0257, 0.0573)	0.2417	(0.0198, 0.1403)	0.0050
Random Forest (RF)	0.6186	0.6676	0.6180	(NA, NA)	NA	(0.0375, 0.1561)	0.0010
Majority Class (MC)	0.5217	0.5000	0.3429	(NA, NA)	NA	(NA, NA)	NA

```
latex_table |>
  kable(digits = 4, format = "latex", booktabs = TRUE, escape = FALSE) |>
  print()
```

```

\begin{tabular}{lrrrrllll}
\toprule
Model & Accuracy & AUC & Macro_F1 & ci_RF & p_RF & ci_MC & p_MC\\
\midrule
Ridge Logistic (RL) & 0.6087 & 0.6513 & 0.6084 &  $\$(-0.0277, 0.0474)\$$  & 0.3241 &  $\$(0.0257, 0.1482)\$$  & 0.0027\\
Lasso Logistic (LL) & 0.6028 & 0.6692 & 0.6023 &  $\$(-0.0257, 0.0573)\$$  & 0.2417 &  $\$(0.0198, 0.1403)\$$  & 0.0050\\
Random Forest (RF) & 0.6186 & 0.6676 & 0.6180 &  $\$(NA, NA)\$$  & NA &  $\$(0.0375, 0.1561)\$$  & 0.0010\\
Majority Class (MC) & 0.5217 & 0.5000 & 0.3429 &  $\$(NA, NA)\$$  & NA &  $\$(NA, NA)\$$  & NA\\
\bottomrule
\end{tabular}

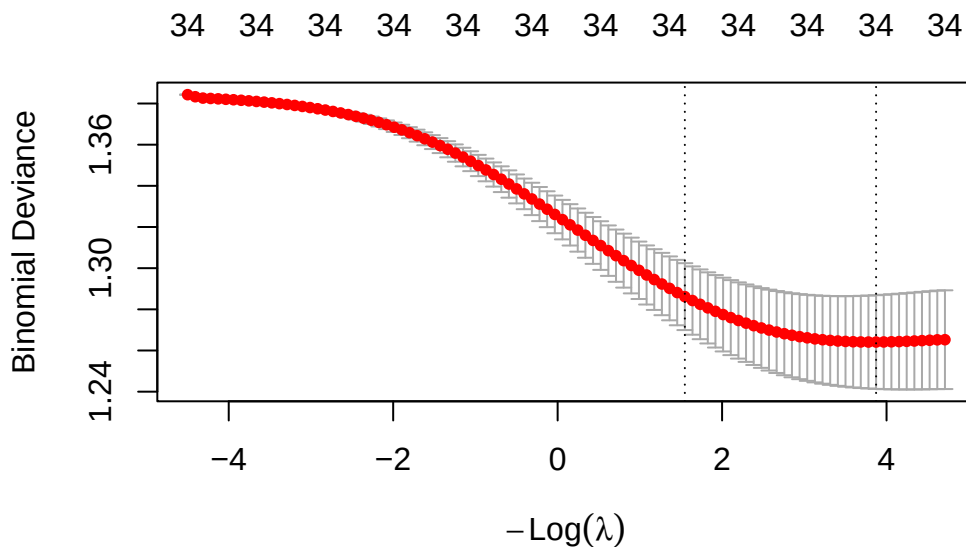
```

Ridge logistic for inference and final model on all data (train + test)

```

# Fit ridge
set.seed(42)
all_bake_no_y <- all_bake |>
  select(-y) |>
  as.matrix()
foldid_all <- caret::createFolds(all_bake$y, k = n_folds, list = FALSE)
cv0_all <- cv.glmnet(x = all_bake_no_y, y = all_bake$y, family = "binomial",
  alpha = 0, foldid = foldid_all)
plot(cv0_all)

```



```
cv0_all
```

```
Call: cv.glmnet(x = all_bake_no_y, y = all_bake$y, foldid = foldid_all, family = "binomial", al
```

```
Measure: Binomial Deviance
```

	Lambda	Index	Measure	SE	Nonzero
min	0.0208	91	1.264	0.02276	34
1se	0.2129	66	1.286	0.01625	34

```
# Bonferroni-corrected confidence intervals for all coefficients; the family
# is the p slopes of this case study (the intercept keeps a marginal CI)
p_slopes <- ncol(all_bake_no_y)
ci_df_all <- logistic_ridge_ci(X = all_bake_no_y,
                              beta_hat = coef(cv0_all, s = cv0_all$lambda.1se),
                              lambda = nrow(all_bake) * cv0_all$lambda.1se / 2,
                              level = level, m = p_slopes)

# Effect sizes as percent changes in odds, exported to CSV for the paper
ci_df_all <- ci_df_all |>
  mutate(
    pct_change = (exp(estimate) - 1) * 100,
    pct_low = (exp(ci_low) - 1) * 100,
    pct_up = (exp(ci_up) - 1) * 100
  )
dir.create("results/final_model", showWarnings = FALSE, recursive = TRUE)
write.csv(ci_df_all, paste0("results/final_model/ci_df_Experiment_",
                           experiment, ".csv"),
          row.names = FALSE)

# Significant slopes with and without the Bonferroni correction, for reference
slopes_df <- ci_df_all |>
  filter(term != "(Intercept)")
zcrit_marg <- qnorm(1 - (1 - level) / 2)
data.frame(
  Rule = c("Unadjusted (marginal CI excludes 0)",
           "Bonferroni (simultaneous CI excludes 0)"),
  N_selected = c(sum(abs(slopes_df$z_value) > zcrit_marg),
                 sum(sign(slopes_df$ci_low * slopes_df$ci_up) > 0)),
  N_total = nrow(slopes_df)
) |>
  kable()
```

Rule	N_selected	N_total
Unadjusted (marginal CI excludes 0)	17	34
Bonferroni (simultaneous CI excludes 0)	9	34

```
# Filter significant slopes (corrected CI excluding zero), order by effect size
ci_df <- slopes_df |>
  filter(sign(ci_low * ci_up) > 0) |>
  mutate(
    term = factor(term, levels = term[order(pct_change, decreasing = FALSE)])
```

```

    )
  )
ci_df |>
  select(term, pct_change, pct_low, pct_up, p_value, p_value_adj) |>
  arrange(desc(pct_change)) |>
  kable(digits = 4)

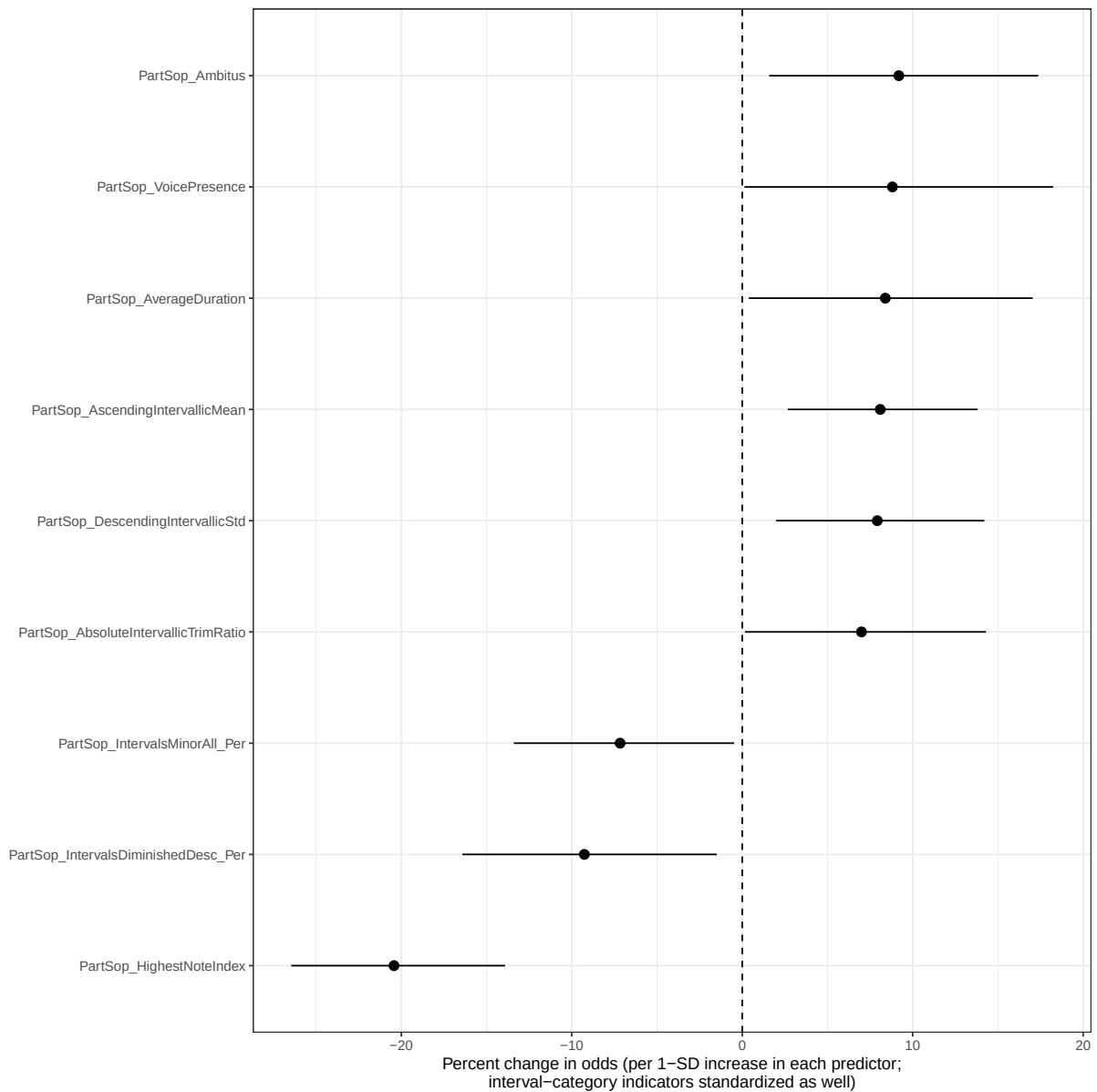
```

term	pct_change	pct_low	pct_up	p_value	p_value_adj
PartSop_Ambitus	9.1882	1.5801	17.3660	0.0001	0.0037
PartSop_VoicePresence	8.8090	0.1328	18.2369	0.0012	0.0419
PartSop_AverageDuration	8.3931	0.3817	17.0438	0.0008	0.0287
PartSop_AscendingIntervallicMean	8.0957	2.6648	13.8140	0.0000	0.0001
PartSop_DescendingIntervallicStd	7.9207	1.9829	14.2042	0.0000	0.0006
PartSop_AbsoluteIntervallicTrimRatio	6.9960	0.1591	14.2996	0.0011	0.0383
PartSop_IntervalsMinorAll_Per	-7.1640	-13.3993	-0.4799	0.0007	0.0229
PartSop_IntervalsDiminished-Desc_Per	-9.2636	-16.4225	-1.4915	0.0002	0.0057
PartSop_HighestNoteIndex	-20.4310	-26.4571	-13.9111	0.0000	0.0000

```

# Plot of coefficients in odds
plot_RL <- ggplot(ci_df, aes(x = term, y = pct_change,
                             ymin = pct_low, ymax = pct_up)) +
  geom_pointrange() +
  geom_hline(yintercept = 0, linetype = 2) +
  coord_flip() +
  labs(
    x = NULL,
    y = paste0("Percent change in odds (per 1-SD increase in each predictor;",
               "\ninterval-category indicators standardized as well)")
  ) +
  theme_bw()
plot_RL

```



```
# Save plot
dir.create("plots", showWarnings = FALSE, recursive = TRUE)
plot_name <- paste0("plots/plot_rl_", experiment, ".png")
ggsave(filename = plot_name, plot = plot_RL, width = 7.5, height = 6,
        dpi = 300)

# Metrics on all data available (in-sample), just for reference
y_all_hat_RL <- predict(cv0_all, newx = all_bake_no_y, s = "lambda.1se",
                        type = "class")
p_all_hat_RL <- predict(cv0_all, newx = all_bake_no_y, s = "lambda.1se",
```

```

      type = "response")
acc_all_RL <- mean(all_bake$y == y_all_hat_RL)
auc_all_RL <- auc_binary(all_bake$y, p_all_hat_RL)
f1_all_RL <- f1(all_bake$y, y_all_hat_RL)
glmnet_summary <- data.frame("Accuracy" = acc_all_RL, "AUC" = auc_all_RL,
                             "Macro_F1" = f1_all_RL,
                             "Model" = "Ridge Logistic (RL)",
                             "Evaluation" = "In-sample (train + test)")

glmnet_summary |>
  kable(digits = 4)

```

Accuracy	AUC	Macro_F1	Model	Evaluation
0.6278	0.6961	0.6266	Ridge Logistic (RL)	In-sample (train + test)

```

# Save results
dir.create("results/final_model", showWarnings = FALSE, recursive = TRUE)
write.csv(glmnet_summary, paste0("results/final_model/RL_final_model_results",
                                "_Experiment_", experiment, ".csv"),
          row.names = FALSE)

```